

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN**



**BÁO CÁO BÀI THỰC HÀNH
HỌC PHẦN: An toàn ứng dụng web và cơ sở dữ liệu
MÃ HỌC PHẦN: INT1405**

**BÀI THỰC HÀNH
Khai thác lỗ hổng Deserialization, RCE và SSTI**

Sinh viên thực hiện:

B23DCAT018 Phan Anh

Giảng viên hướng dẫn: ThS. Ninh Thị Thu Trang

HỌC KỲ 1 NĂM HỌC 2026-2027

MỤC LỤC

MỤC LỤC	2
DANH MỤC CÁC HÌNH VẼ	3
CHƯƠNG 1. Khai thác lỗ hổng Deserialization, RCE và SSTI	4
1.1 Giới thiệu Lab	4
1.2 Kịch bản	4
1.3 Kiến thức bổ trợ	5
1.4 Các nhiệm vụ của bài thực hành	6

DANH MỤC CÁC HÌNH VẼ

<i>Hình 1</i> Kiểm tra kết nối.....	7
<i>Hình 2</i> Truy cập giao diện OWASP Juice Shop.....	8
<i>Hình 3</i> Khởi động OWASP ZAP.....	8
<i>Hình 4</i> Bấm Close.....	9
<i>Hình 5</i> Mở Preferences.....	10
<i>Hình 6</i> Tìm kiếm proxy.....	10
<i>Hình 7</i> Chọn Settings.....	11
<i>Hình 8</i> Xem chi tiết chứng chỉ SSL.....	12
<i>Hình 9</i> Bấm chọn Confirm Security Exception.....	12
<i>Hình 10</i> Firefox truy cập được ứng dụng web qua proxy của OWASP ZAP.....	13
<i>Hình 11</i> Bắt đầu quét.....	14
<i>Hình 12</i> Kết quả quét.....	16
<i>Hình 13</i> Lưu kết quả.....	17
<i>Hình 14</i> Xem chi tiết POST.....	18
<i>Hình 15</i> Bấm thử Try it out.....	18
<i>Hình 16</i> Sao chép trường Authorization.....	19
<i>Hình 17</i> Dán Bearer Token.....	19
<i>Hình 18</i> API được xác thực thành công.....	20
<i>Hình 19</i> Execute thành công.....	20
<i>Hình 20</i> Thử redos.....	21
<i>Hình 21</i> Trang profile.....	22
<i>Hình 22</i> Set thử Username.....	23
<i>Hình 23</i> Set Username thành #{1+1}.....	24
<i>Hình 24</i> Xác định được flag.....	25
<i>Hình 25</i> Checkwork hoàn thành bài lab.....	26

CHƯƠNG 1. KHAI THÁC LỖ HỔNG DESERIALIZATION, RCE VÀ SSTI

1.1 Giới thiệu Lab

Thông tin	Chi tiết
Tên Lab	web-insdes
Mức độ	Trung bình
Công cụ sử dụng	OWASP ZAP, Firefox, Swagger API, Terminal (Linux)

Mục tiêu học tập

Bài thực hành nhằm rèn luyện kỹ năng khai thác lỗ hổng Insecure Deserialization trên ứng dụng web. Thông qua bài lab, sinh viên sử dụng OWASP ZAP để chặn bắt và chỉnh sửa các yêu cầu POST định dạng JSON gửi tới API B2B của ứng dụng, từ đó buộc máy chủ thực thi các đoạn mã ngoài ý muốn, gây từ chối dịch vụ (DoS) và cuối cùng khai thác lỗ hổng Server-Side Template Injection (SSTI) trên trang hồ sơ người dùng.

Sử dụng OWASP ZAP và Swagger API để khảo sát và thao tác trực tiếp với API B2B của ứng dụng.

Khai thác lỗ hổng Insecure Deserialization để buộc máy chủ thực thi mã JavaScript tùy ý thông qua trường dữ liệu JSON.

Gây từ chối dịch vụ (DoS) bằng kỹ thuật ReDoS (Regular Expression Denial of Service).

Khai thác lỗ hổng Server-Side Template Injection (SSTI) trên trang hồ sơ người dùng để đọc dữ liệu nhạy cảm trên máy chủ.

Kỹ năng đạt được

Kỹ năng sử dụng OWASP ZAP kết hợp Swagger UI để phân tích và chỉnh sửa trực tiếp HTTP Request dạng JSON.

Kỹ năng nhận diện và khai thác lỗ hổng Insecure Deserialization dẫn đến thực thi mã từ xa (RCE).

Kỹ năng nhận diện Template Engine của ứng dụng và khai thác lỗ hổng SSTI.

Kỹ năng đánh giá rủi ro và đề xuất biện pháp khắc phục cho các lỗ hổng thuộc nhóm Insecure Deserialization và Injection.

1.2 Kịch bản

Tình huống:

Ứng dụng thương mại điện tử OWASP Juice Shop cung cấp một API B2B cho phép đối tác

gửi đơn hàng dưới định dạng JSON. Đội ngũ phát triển lo ngại rằng cơ chế xử lý dữ liệu đầu vào tại API này chưa được kiểm soát chặt chẽ, có thể cho phép kẻ tấn công chèn mã thực thi tùy ý thông qua quá trình deserialization. Bên cạnh đó, trang hồ sơ người dùng được xây dựng bằng một Template Engine cũng tiềm ẩn nguy cơ bị khai thác SSTI. Sinh viên đóng vai trò một Chuyên gia đánh giá bảo mật (Security Auditor), được cấp quyền truy cập như người dùng thông thường và cần xác minh, khai thác các lỗ hổng nêu trên để đánh giá mức độ rủi ro của hệ thống.

Vai trò của sinh viên:

Dùng OWASP ZAP làm proxy trung gian để chặn bắt lưu lượng HTTP/HTTPS giữa Firefox và máy chủ mục tiêu.

Thao tác trực tiếp với API B2B qua Swagger UI để khai thác lỗ hổng Insecure Deserialization, gây thực thi mã và từ chối dịch vụ trên máy chủ.

Khai thác lỗ hổng SSTI trên trang hồ sơ người dùng để đọc dữ liệu nhạy cảm được lưu trữ trên máy chủ.

Sơ đồ mạng



1.3 Kiến thức bổ trợ

Insecure Deserialization

Insecure Deserialization xảy ra khi ứng dụng khôi phục (deserialize) dữ liệu từ một chuỗi JSON hoặc XML mà không kiểm tra kỹ nội dung bên trong, dẫn đến việc các đoạn mã hoặc lệnh gọi hàm được nhúng trong dữ liệu đó bị máy chủ thực thi. Trong bài lab này, API B2B của Juice Shop nhận trường orderLinesData dưới dạng chuỗi và thực thi trực tiếp nội dung bên trong như một biểu thức JavaScript, cho phép sinh viên kiểm chứng lỗ hổng bằng cách gửi các payload gây vòng lặp vô hạn hoặc biểu thức chính quy tốn tài nguyên (ReDoS).

Server-Side Template Injection (SSTI)

SSTI xảy ra khi dữ liệu người dùng nhập vào được Template Engine (ở đây là Pug) xử lý như một biểu thức thay vì một chuỗi văn bản thông thường. Nếu ứng dụng không phân tách rõ dữ liệu và mã lệnh, kẻ tấn công có thể chèn biểu thức Pug để buộc máy chủ thực thi mã JavaScript tùy ý, từ đó đọc file hệ thống hoặc chiếm quyền điều khiển máy chủ.

Công cụ sử dụng trong bài

Công cụ / Chức năng	Mô tả tác dụng
Spider (OWASP ZAP)	Tự động thu thập URL, tài nguyên của ứng dụng để phát hiện các endpoint không hiển thị trực tiếp trên giao diện, ví dụ tài liệu Swagger API.
Swagger UI	Giao diện trực quan để xem tài liệu và gửi trực tiếp các yêu cầu tới API B2B, phục vụ chỉnh sửa Request Body dạng JSON.
Pug Template Engine	Template Engine dựng trang hồ sơ người dùng của Juice Shop, là vị trí tồn tại lỗ hổng SSTI được khai thác trong bài lab.

1.4 Các nhiệm vụ của bài thực hành

Trước khi thực hiện bài thực hành, sinh viên thực hiện chạy câu lệnh:

```
imodule
```

https://raw.githubusercontent.com/thanhdat317/Imodule_labtainer/master/web-insdes.tar

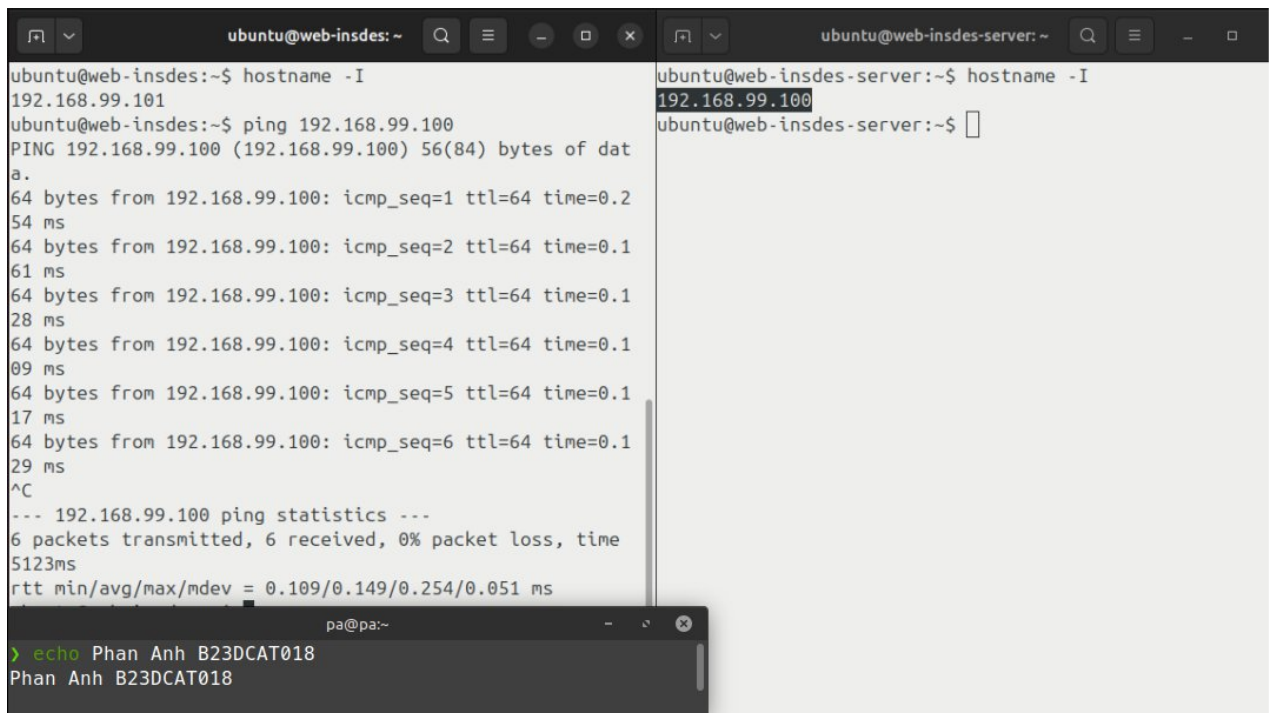
Để khởi động bài thực hành, sinh viên dùng câu lệnh:

```
labtainer web-insdes
```

Username	Password	Hệ thống
admin@juice.org	admin123	Tài khoản quản trị để đăng nhập web
jim@juice.org	ncc-1701	Đăng nhập web
Ubuntu	ubuntu	Máy trạm sinh viên và máy chủ chứa lỗ hổng

Nhiệm vụ 1 - Kiểm tra kết nối tới máy chủ web

Tại giao diện dòng lệnh máy sinh viên (web-insdes) thực hiện kiểm tra kết nối tới máy chủ web.



The image shows two terminal windows side-by-side. The left window is titled 'ubuntu@web-insdes: ~' and shows the output of 'hostname -I' (192.168.99.101) and a 'ping 192.168.99.100' command. The ping results show 6 successful packets with varying times. The right window is titled 'ubuntu@web-insdes-server: ~' and shows the output of 'hostname -I' (192.168.99.100). A third, smaller terminal window is overlaid at the bottom left, titled 'pa@pa:~', showing the command 'echo Phan Anh B23DCAT018' and its output.

```
ubuntu@web-insdes:~$ hostname -I
192.168.99.101
ubuntu@web-insdes:~$ ping 192.168.99.100
PING 192.168.99.100 (192.168.99.100) 56(84) bytes of data:
64 bytes from 192.168.99.100: icmp_seq=1 ttl=64 time=0.254 ms
64 bytes from 192.168.99.100: icmp_seq=2 ttl=64 time=0.161 ms
64 bytes from 192.168.99.100: icmp_seq=3 ttl=64 time=0.128 ms
64 bytes from 192.168.99.100: icmp_seq=4 ttl=64 time=0.109 ms
64 bytes from 192.168.99.100: icmp_seq=5 ttl=64 time=0.117 ms
64 bytes from 192.168.99.100: icmp_seq=6 ttl=64 time=0.129 ms
^C
--- 192.168.99.100 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time
5123ms
rtt min/avg/max/mdev = 0.109/0.149/0.254/0.051 ms

pa@pa:~$ echo Phan Anh B23DCAT018
Phan Anh B23DCAT018
```

```
ubuntu@web-insdes-server:~$ hostname -I
192.168.99.100
ubuntu@web-insdes-server:~$
```

Hình 1 Kiểm tra kết nối

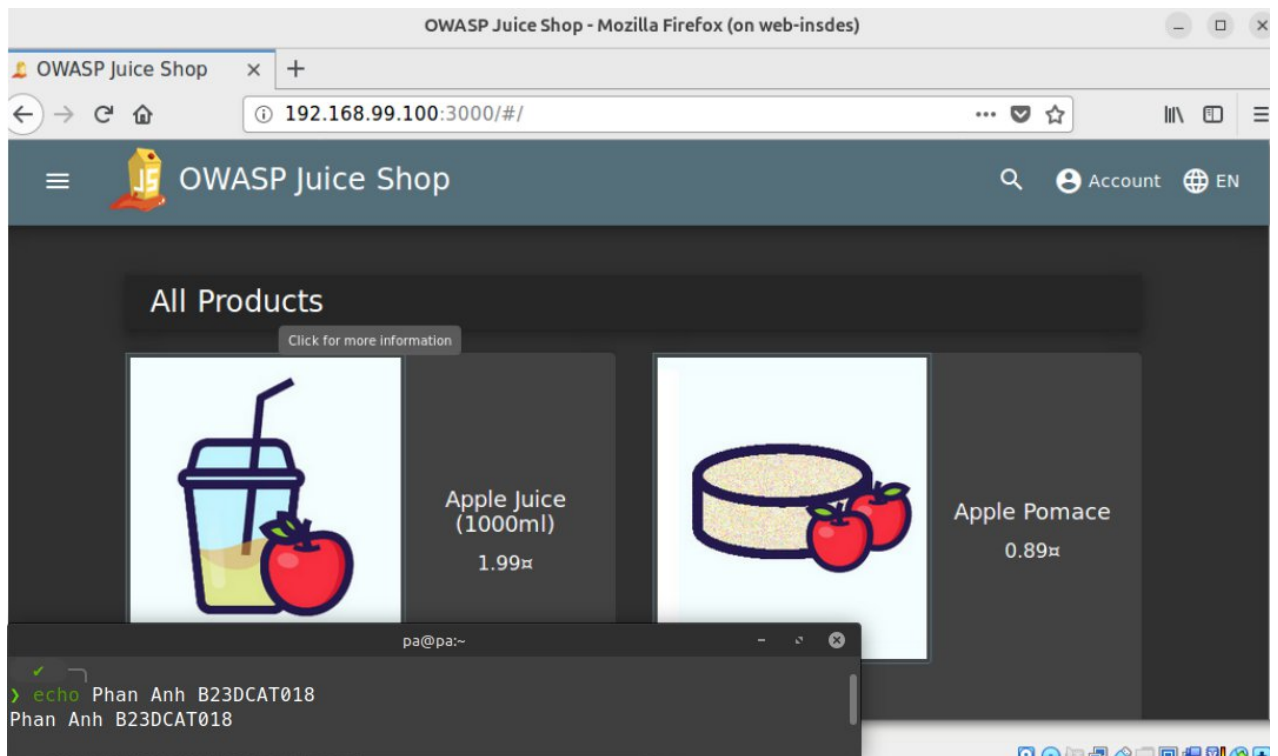
Nhận được phản hồi từ máy chủ web, xác nhận máy trạm và máy chủ web có thể liên lạc được với nhau.

Nhiệm vụ 2 - Mở Firefox và truy cập vào máy chủ web

Bước 1: Tại giao diện dòng lệnh máy sinh viên, gõ lệnh sau để mở trình duyệt Firefox:
firefox &

Bước 2: Trên Firefox, truy cập đường dẫn của ứng dụng web:

<http://192.168.99.100:3000>



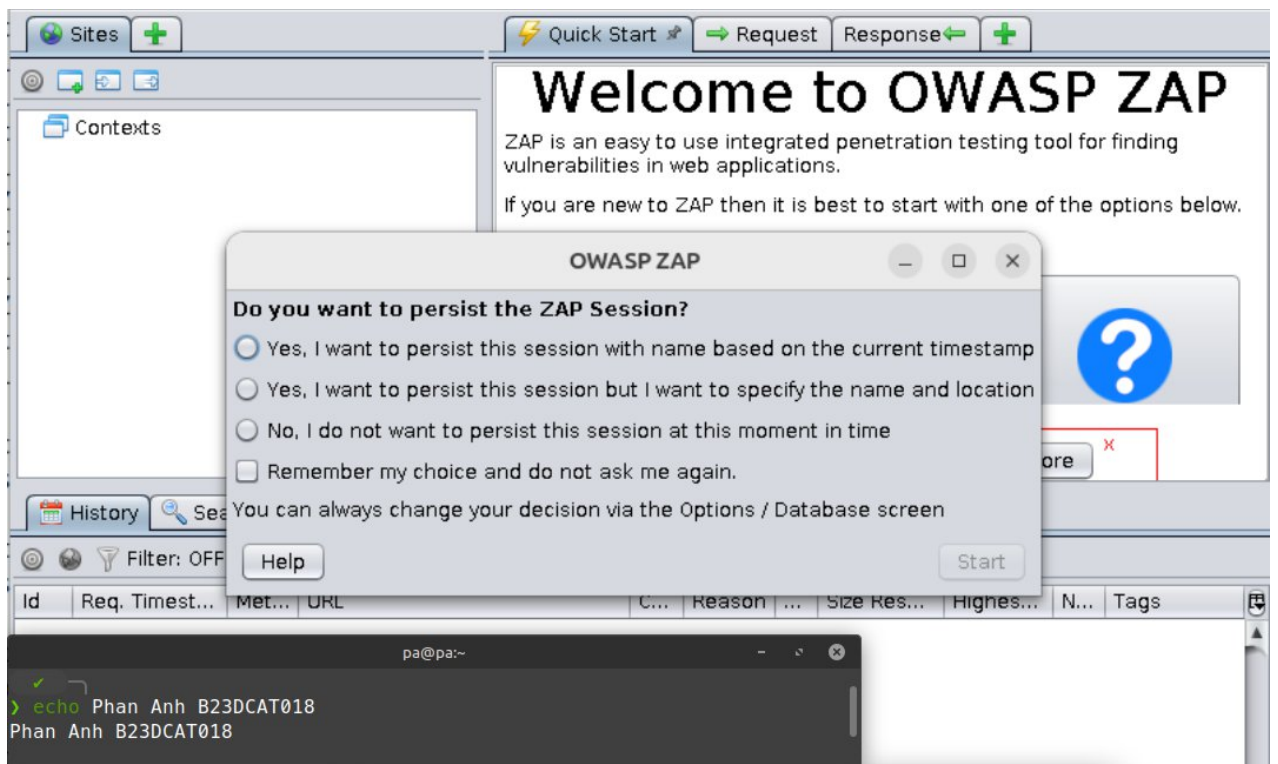
Hình 2 Truy cập giao diện OWASP Juice Shop

Truy cập thành công giao diện của ứng dụng OWASP Juice Shop trên Firefox.

Nhiệm vụ 3 - Khởi động và cài đặt OWASP ZAP

Bước 1: Tại giao diện dòng lệnh, khởi động OWASP ZAP:

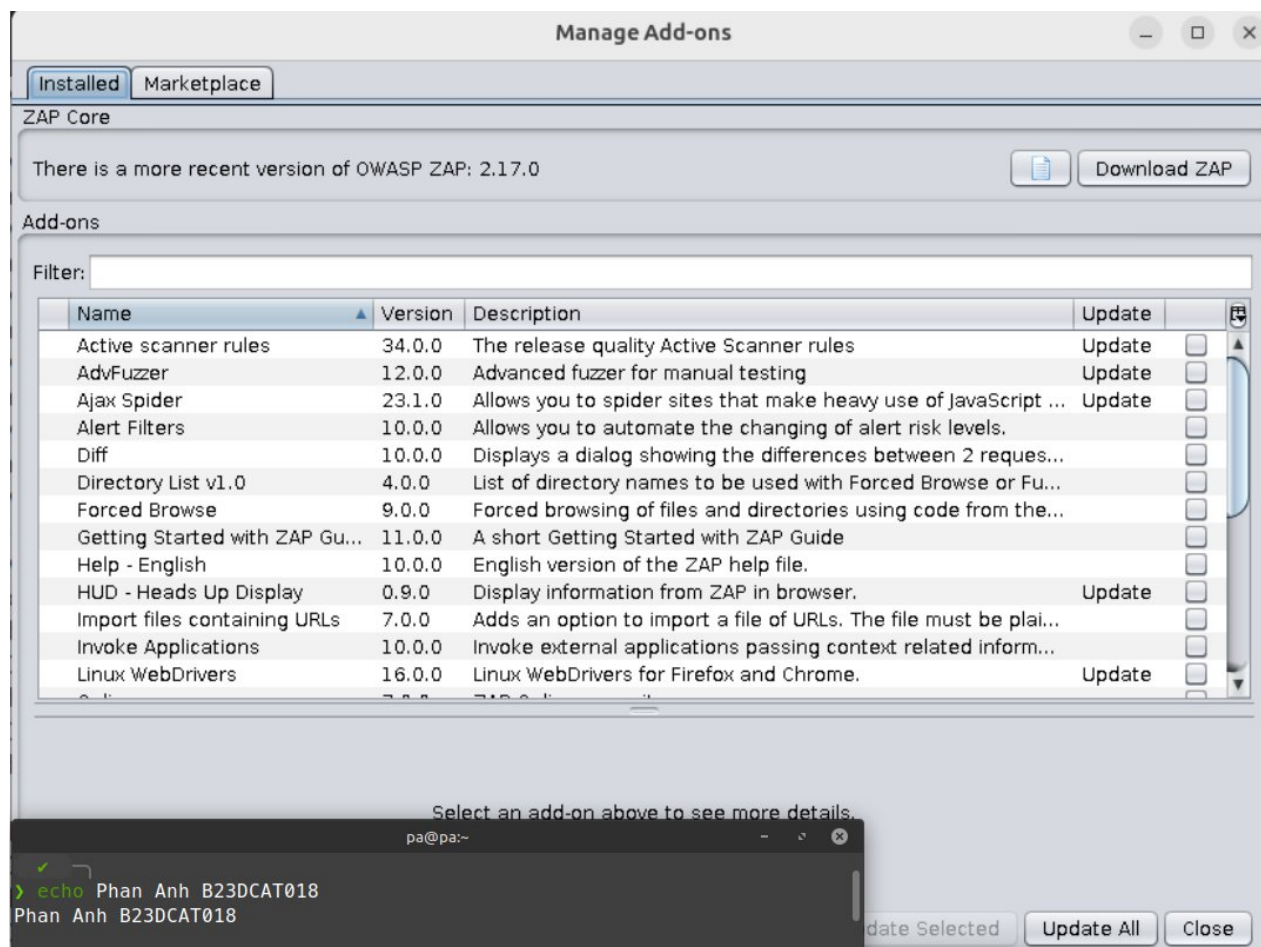
owasp-zap &



Hình 3 Khởi động OWASP ZAP

Bước 2: Khi OWASP ZAP mở lên và yêu cầu nhập thông tin, sinh viên chọn "yes, I want to persist this session with the name based on the current timestamp" rồi chọn Start.

Bước 3: Nếu xuất hiện hộp thoại "Manage Add-on", sinh viên chọn Close để tiếp tục.



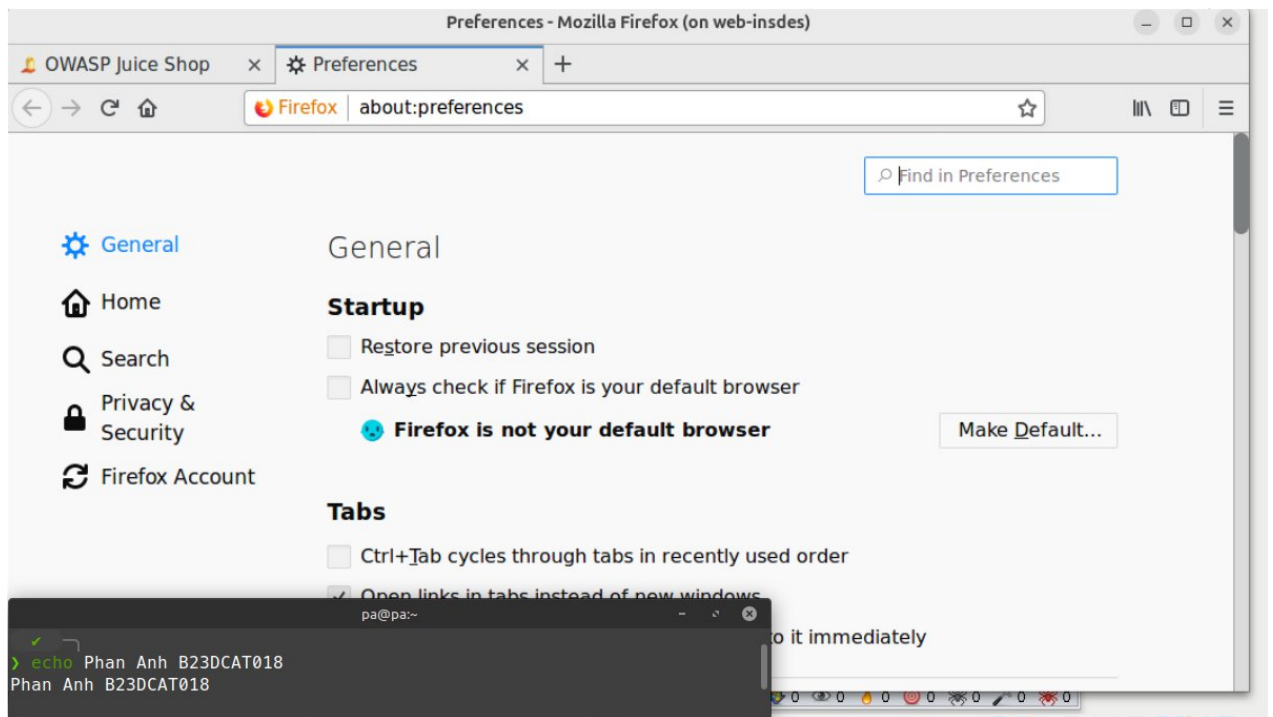
Hình 4 Bấm Close

OWASP ZAP khởi động thành công và sẵn sàng để cấu hình làm proxy.

Nhiệm vụ 4 - Cấu hình Firefox Proxy để sử dụng OWASP ZAP

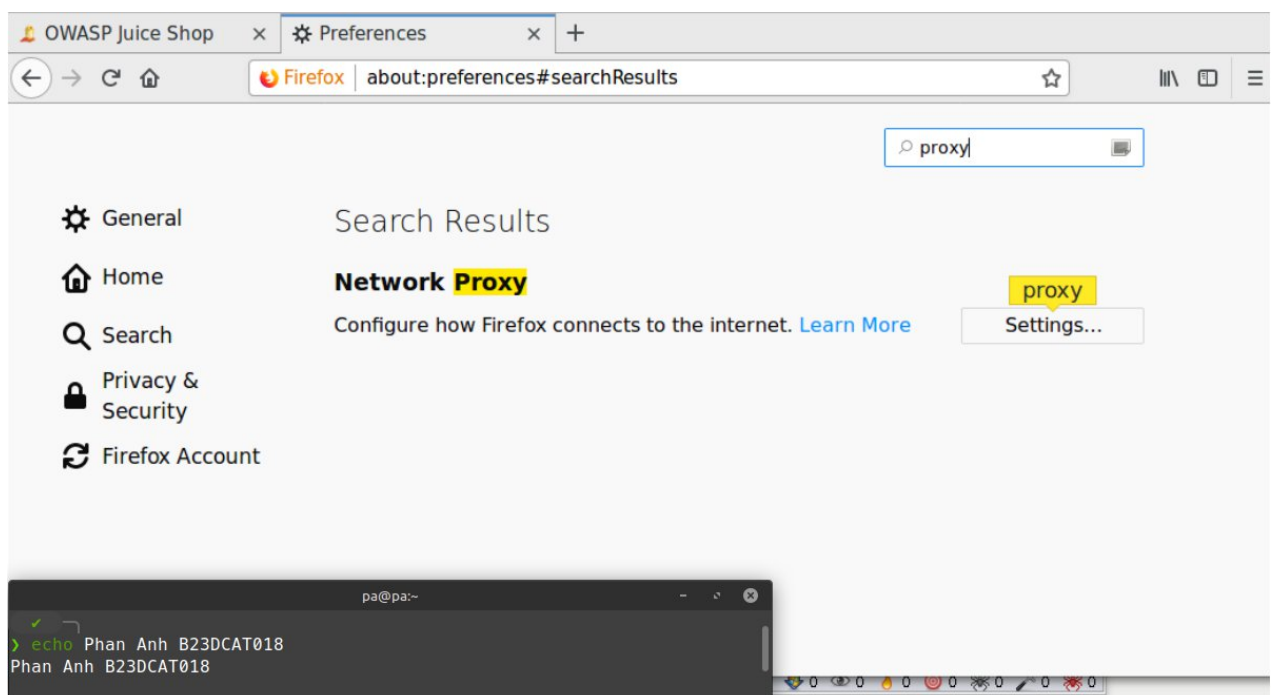
Mục tiêu: Cấu hình OWASP ZAP làm proxy trung gian để bắt và phân tích các gói tin HTTP/HTTPS truyền từ trình duyệt Firefox trên máy trạm của sinh viên. Thực hiện cấu hình trên trình duyệt Firefox theo các bước sau đây:

Bước 1: Trên Firefox của máy tính sinh viên, mở Preferences.



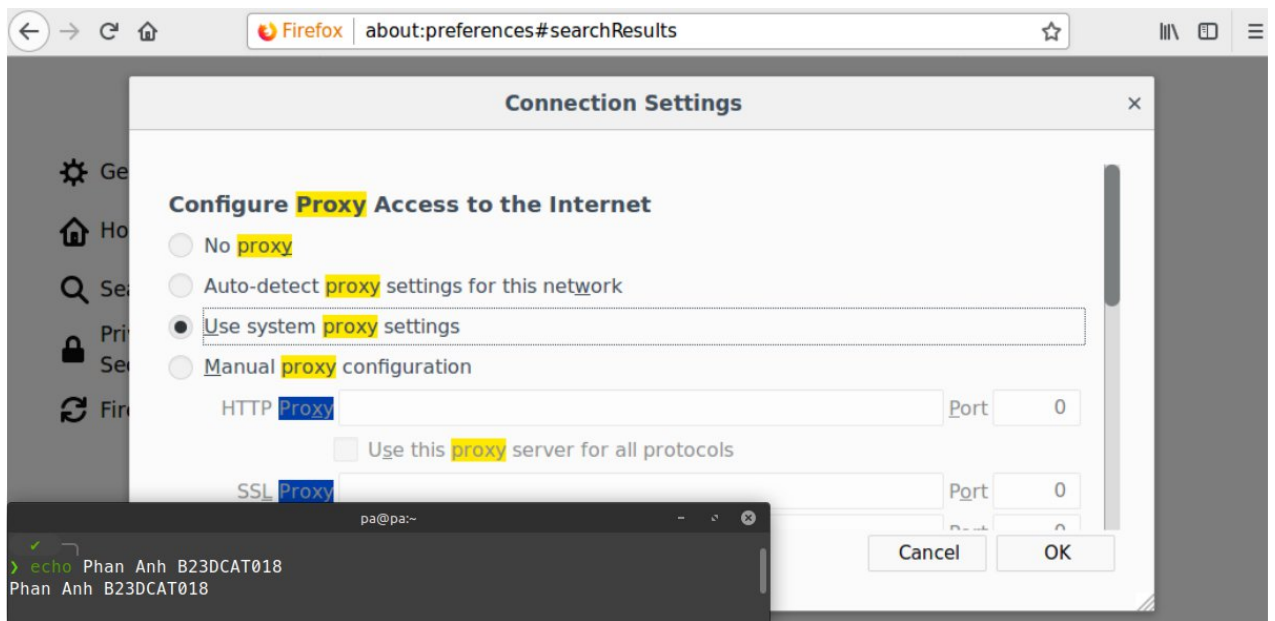
Hình 5 Mở Preferences

Bước 2: Trong ô tìm kiếm cài đặt, gõ proxy.



Hình 6 Tìm kiếm proxy

Bước 3: Trong mục Network Proxy, chọn Settings.



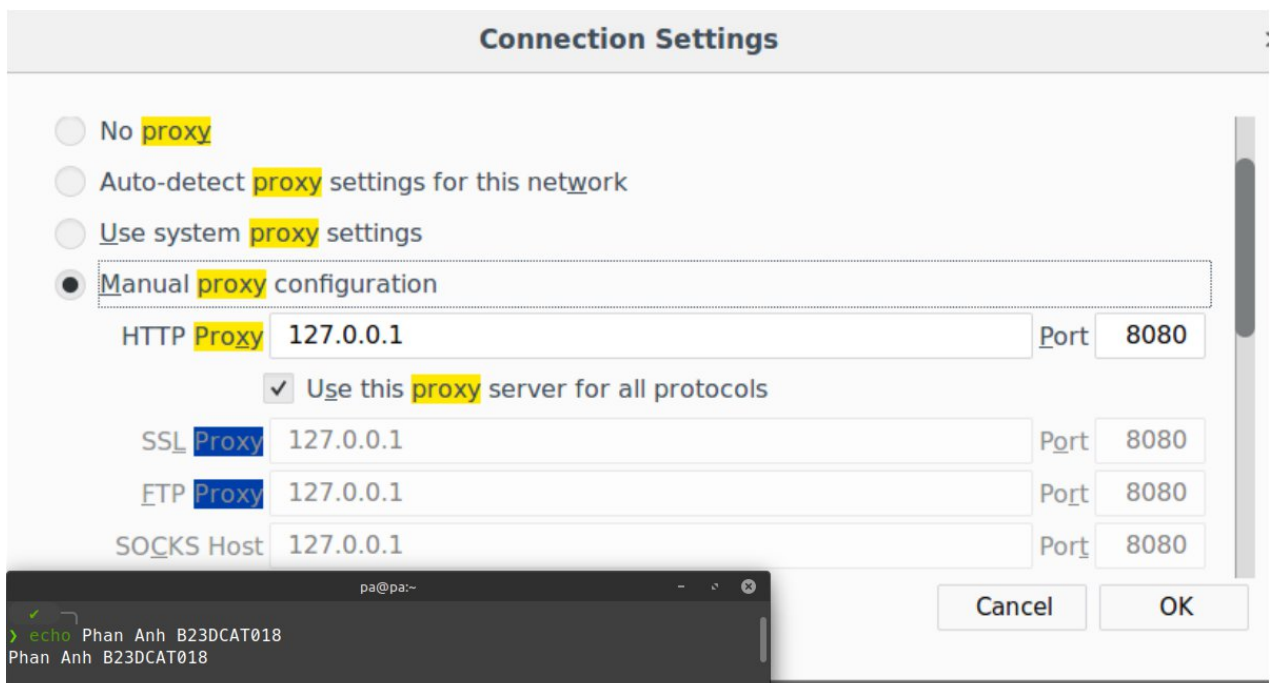
Hình 7 Chọn Settings

Bước 4: Chọn Manual proxy configuration (Cấu hình proxy thủ công).

Bước 5: Tại trường HTTP Proxy, nhập 127.0.0.1, Port nhập 8080.

Bước 6: Chọn thêm Use this proxy server for all protocols (Sử dụng máy chủ proxy này cho tất cả các giao thức).

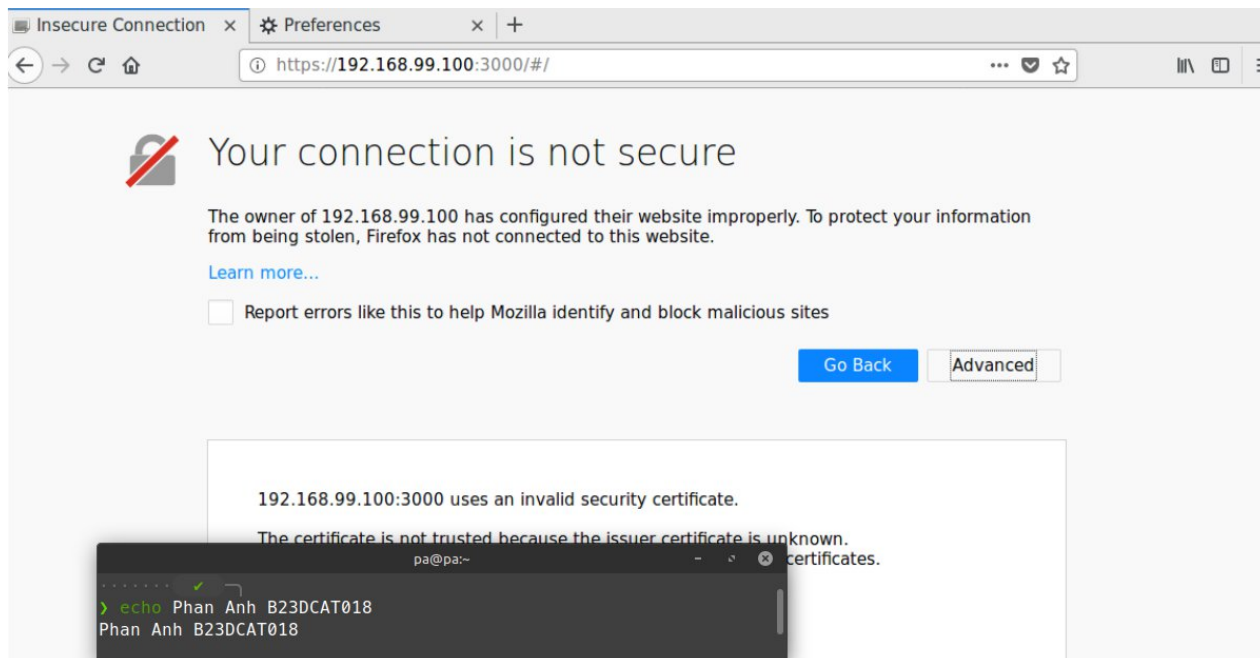
Bước 7: Chọn OK để chấp nhận các cài đặt.



Sau khi cấu hình xong, thực hiện các bước sau để xác nhận Firefox đang sử dụng ZAP làm proxy:

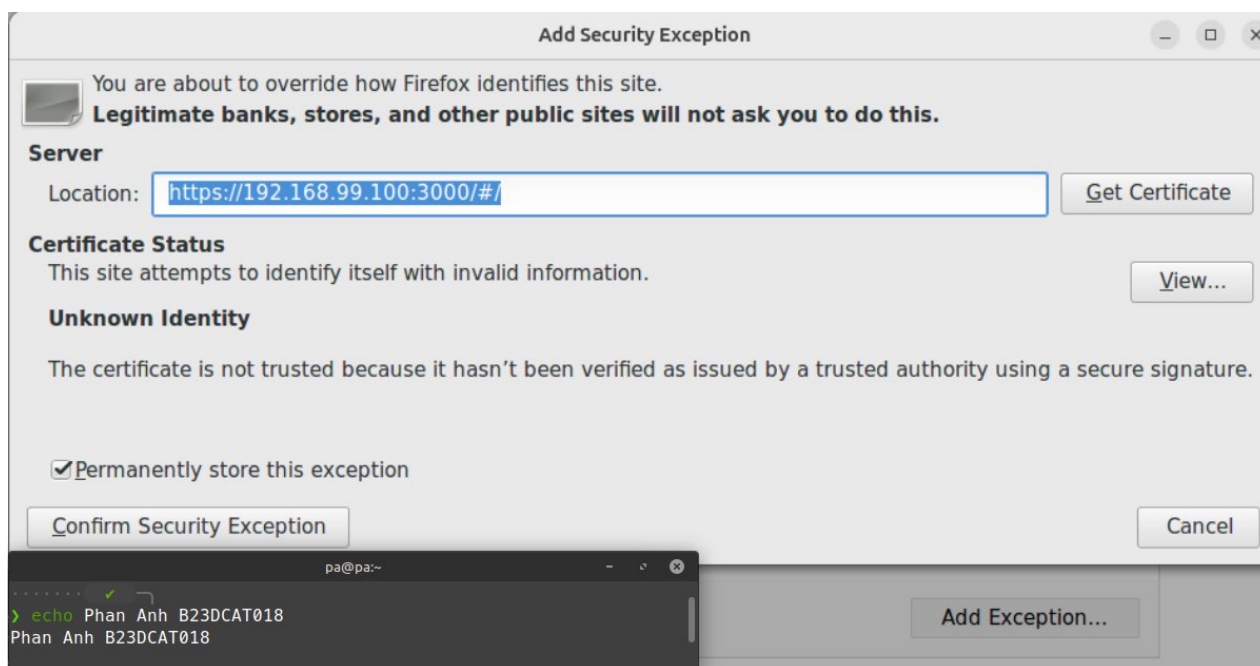
Bước 8: Làm mới trang web 192.168.99.100:3000. Một cảnh báo bảo mật với thông báo "Kết nối của bạn không an toàn" sẽ xuất hiện.

Bước 9: Sinh viên tiếp tục bấm chọn Advanced để xem chi tiết chứng chỉ SSL (thường hiển thị lỗi SEC_ERROR_UNKNOWN_ISSUER), sau đó chọn Add Exception để chấp nhận chứng chỉ này.



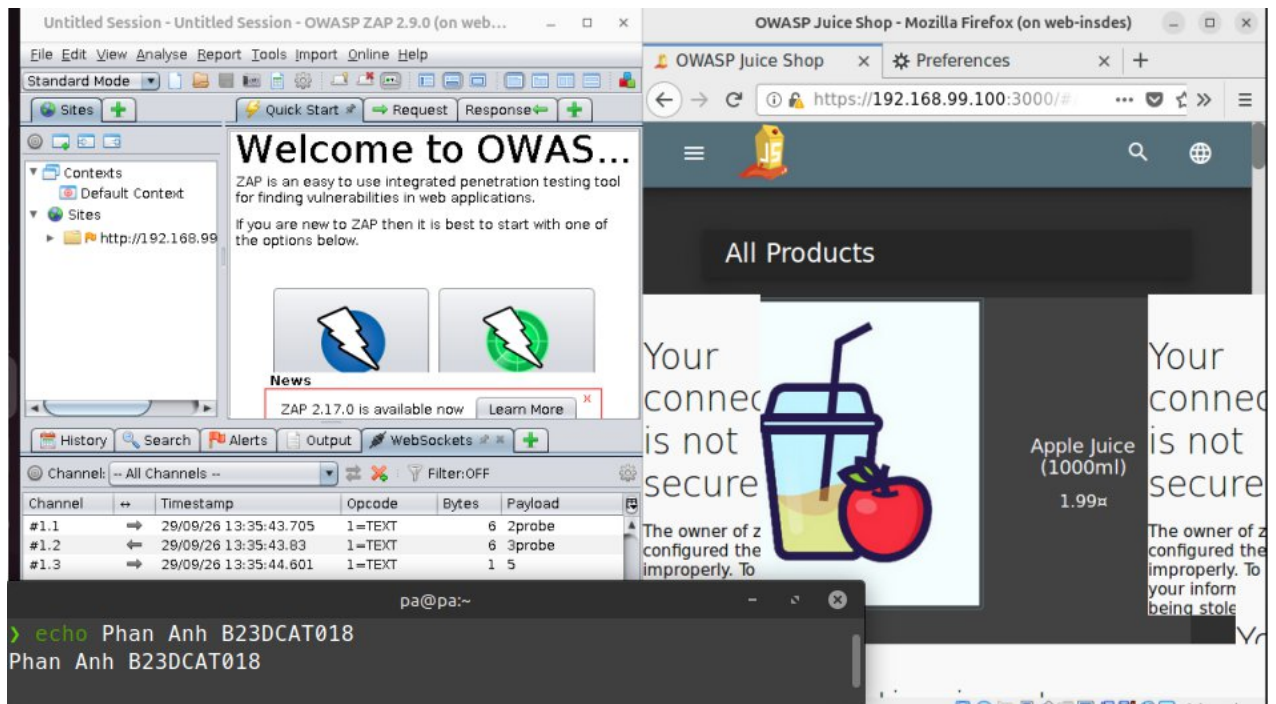
Hình 8 Xem chi tiết chứng chỉ SSL

Bước 10: Tại cửa sổ xác nhận, sinh viên tiếp tục bấm chọn Confirm Security Exception.



Hình 9 Bấm chọn Confirm Security Exception

Firefox truy cập được ứng dụng web qua proxy của OWASP ZAP, tab History trong ZAP bắt đầu ghi nhận các request.

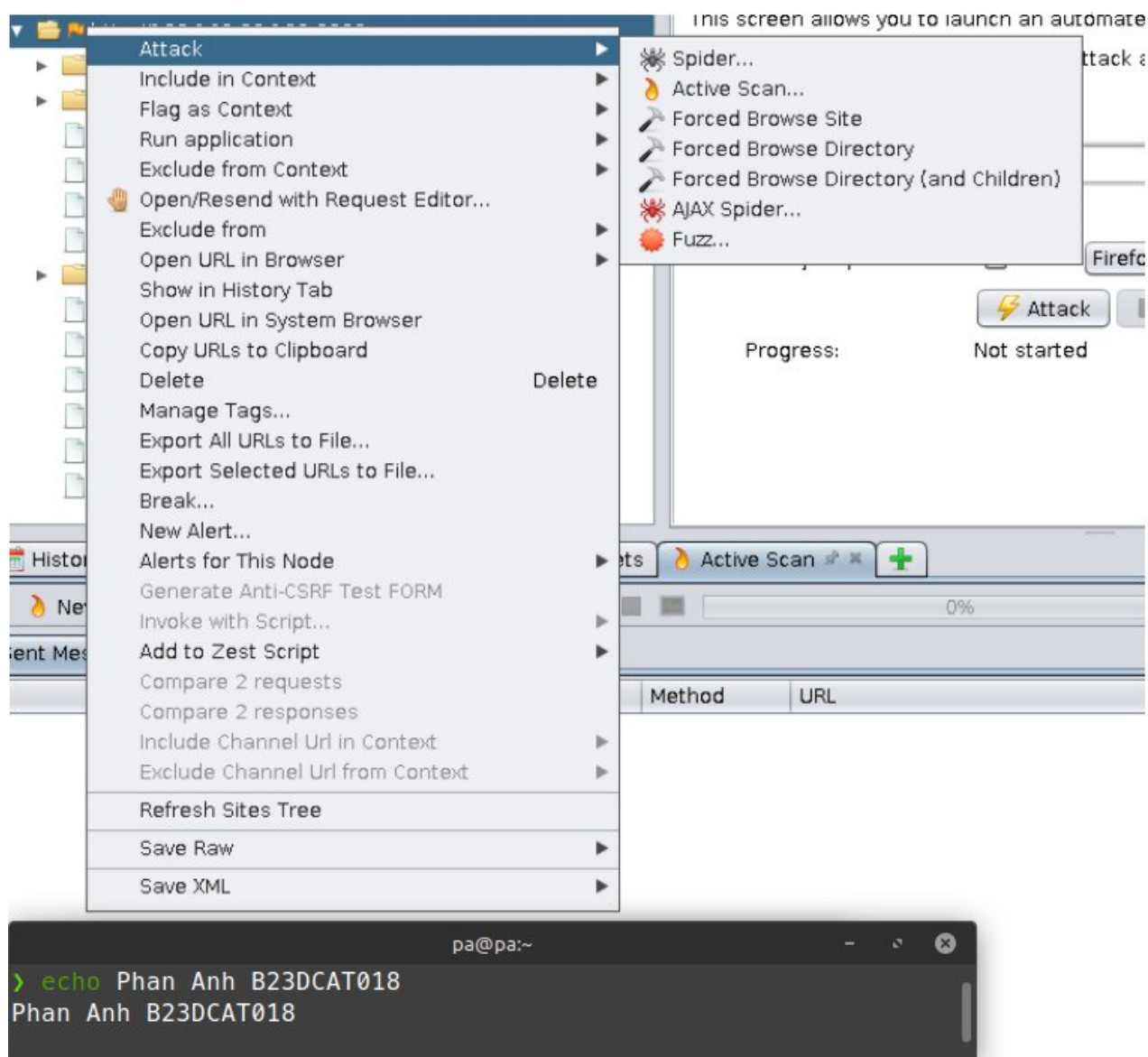


Hình 10 Firefox truy cập được ứng dụng web qua proxy của OWASP ZAP

Nhiệm vụ 5 - Truy cập vào dữ liệu bị hạn chế

Mục tiêu: Sử dụng công cụ Spider của OWASP ZAP để tự động thu thập các URL và tài nguyên của ứng dụng, qua đó xác định những đường dẫn hoặc chức năng không được hiển thị trực tiếp trên giao diện nhưng vẫn có thể được truy cập. Theo OWASP WSTG-ATHZ-01, việc lập danh sách các URL và thư mục của ứng dụng là bước quan trọng trong quá trình kiểm thử, giúp phát hiện các endpoint nhạy cảm bị lộ do cấu hình chưa phù hợp.

Bước 1: Tại tab Sites của OWASP ZAP, chọn trang 192.168.99.100:3000, sau đó vào Attack → Spider để bắt đầu quét.



Hình 11 Bắt đầu quét

Bước 2: Sau khi quét hoàn tất, quan sát danh sách đường dẫn được tìm thấy

Active Scan

Spider

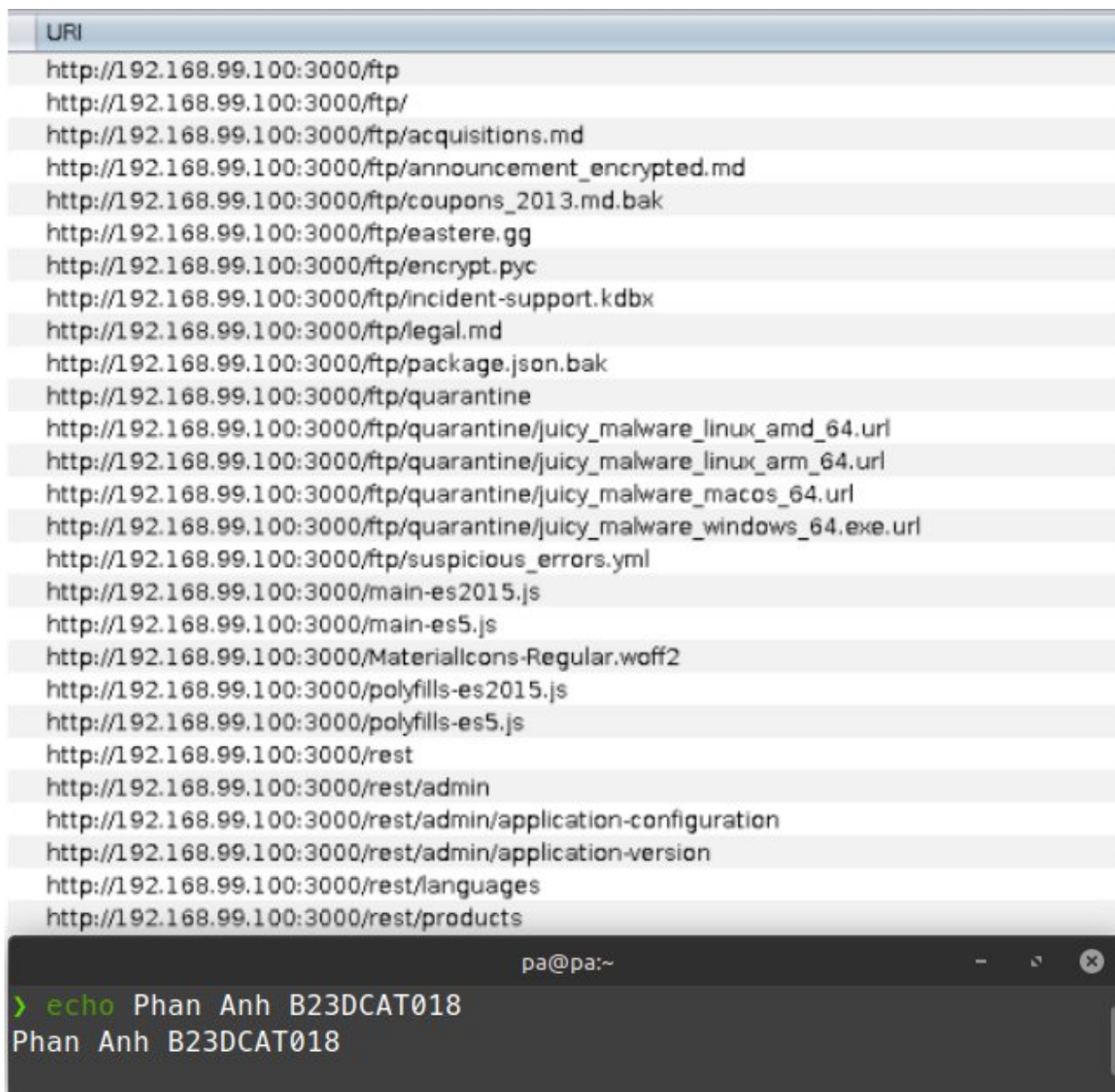
+

100%

Current Scans: 0 | URLs Found: 336 | Nodes Ad

URI
http://192.168.99.100:3000/
http://192.168.99.100:3000/robots.txt
http://192.168.99.100:3000/sitemap.xml
http://192.168.99.100:3000/api
http://192.168.99.100:3000/api/Challenges/?name=Score%20Board
http://192.168.99.100:3000/api/Quantities/
http://192.168.99.100:3000/assets
http://192.168.99.100:3000/assets/i18n
http://192.168.99.100:3000/assets/i18n/en.json
http://192.168.99.100:3000/ftp
http://192.168.99.100:3000/assets/public
http://192.168.99.100:3000/assets/public/favicon.js.ico
http://192.168.99.100:3000/assets/public/images
http://192.168.99.100:3000/assets/public/images/products
http://192.168.99.100:3000/font-mfizz.woff
http://cdnjs.cloudflare.com/ajax/libs/cookieconsent2/3.1.0/cookieconsent.min.css
http://192.168.99.100:3000/main-es2015.js
http://192.168.99.100:3000/styles.css
http://192.168.99.100:3000/MaterialIcons-Regular.woff2
http://192.168.99.100:3000/polyfills-es2015.js
http://192.168.99.100:3000/rest
http://192.168.99.100:3000/rest/admin
http://192.168.99.100:3000/rest/admin/application-configuration
http://192.168.99.100:3000/rest/admin/application-version
http://192.168.99.100:3000/rest/languages
http://cdnjs.cloudflare.com/ajax/libs/cookieconsent2/3.1.0/cookieconsent.min.js
http://cdnjs.cloudflare.com/ajax/libs/jquery/2.2.4/jquery.min.js
http://192.168.99.100:3000/runtime-es2015.js
http://192.168.99.100:3000/rest/products
http://192.168.99.100:3000/runtime-es5.js
http://192.168.99.100:3000/polyfills-es5.js
http://192.168.99.100:3000/rest/products/search

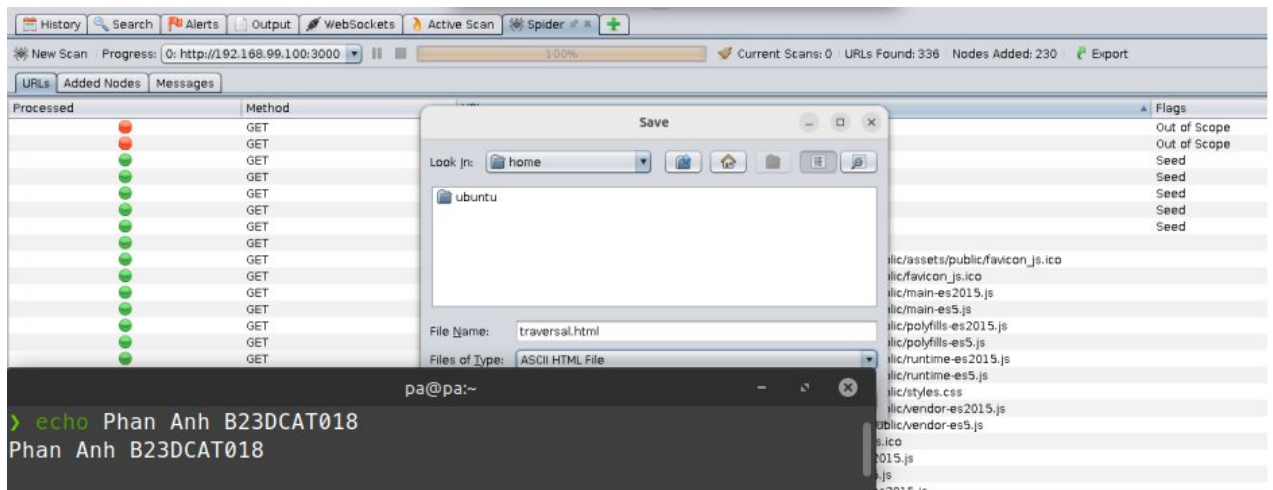
> echo Phan Anh B23DCAT018
 Phan Anh B23DCAT018



Hình 12 Kết quả quét

- ZAP tìm được /ftp.
- Có các file nhạy cảm trong /ftp.
- Có các endpoint /rest/admin, /api, /rest/products.

Bước 3: Lưu kết quả quét dưới dạng báo cáo HTML, lưu tại Desktop với tên traversal.html. Trường hợp không có tùy chọn Traversal, hãy lưu toàn bộ báo cáo.



Hình 13 Lưu kết quả

Xác định được các endpoint quan trọng (tài liệu Swagger API, thư mục ftp) thông qua kết quả Spider và lưu được báo cáo quét.

Câu hỏi 1:

Sinh viên tìm thấy những đường dẫn đáng chú ý nào qua kết quả Spider, và vì sao các đường dẫn này không nên bị lộ với người dùng thông thường?

Trả lời: Qua kết quả Spider của OWASP ZAP, em tìm thấy một số đường dẫn đáng chú ý như /api, /rest, /rest/admin, /rest/admin/application-configuration, /rest/admin/application-version, /rest/products, /api/Challenges, /api/Quantitys và đặc biệt là thư mục /ftp với nhiều tệp như acquisitions.md, announcement_encrypted.md, coupons_2013.md.bak, incident-support.kdbx, legal.md, package.json.bak, encrypt.pyc và thư mục /ftp/quarantine.

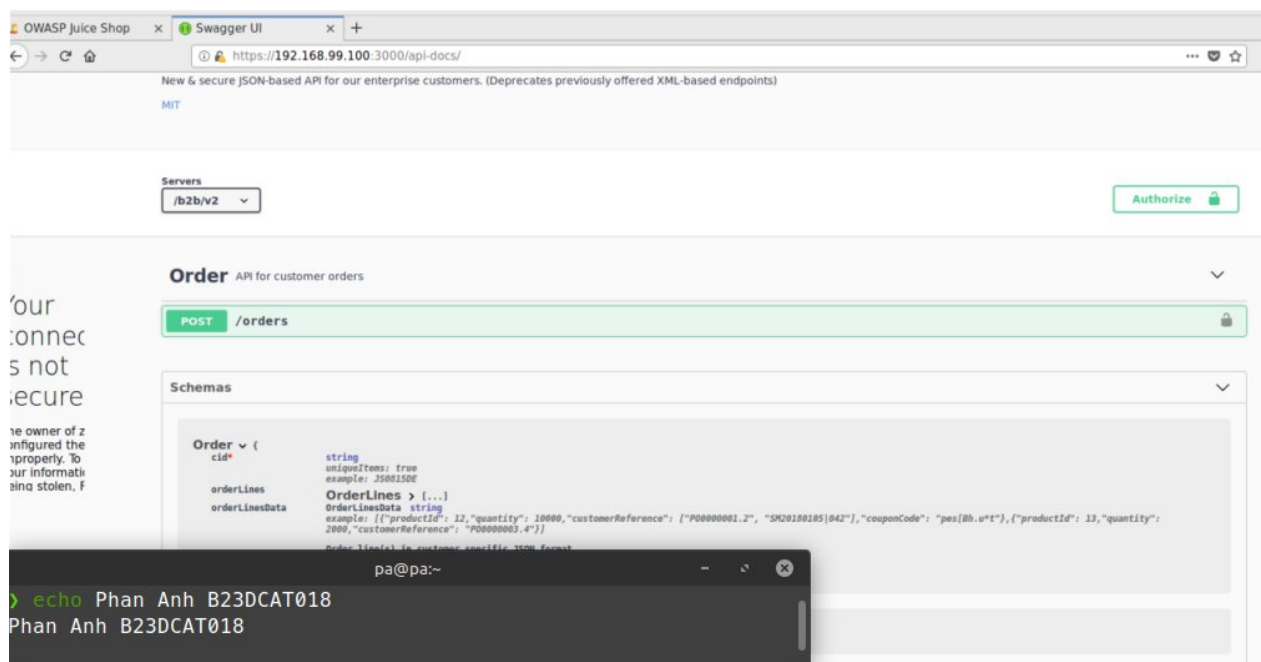
Các đường dẫn này không nên bị lộ với người dùng thông thường vì chúng có thể tiết lộ cấu trúc API, chức năng quản trị, thông tin cấu hình ứng dụng, tài liệu nội bộ hoặc các tệp sao lưu có khả năng chứa dữ liệu nhạy cảm. Nếu attacker biết được các endpoint này, họ có thể tiếp tục phân tích để tìm lỗi phân quyền, truy cập trái phép dữ liệu bị hạn chế hoặc khai thác các chức năng không được bảo vệ đầy đủ. Vì vậy, các endpoint và thư mục này cần được kiểm soát truy cập chặt chẽ, không cho phép người dùng không có thẩm quyền truy cập trực tiếp.

Nhiệm vụ 6 - Khai thác Insecure Deserialization để thực thi mã từ xa (RCE)

6.1 - Khai thác Insecure Deserialization để thực thi mã từ xa (RCE)

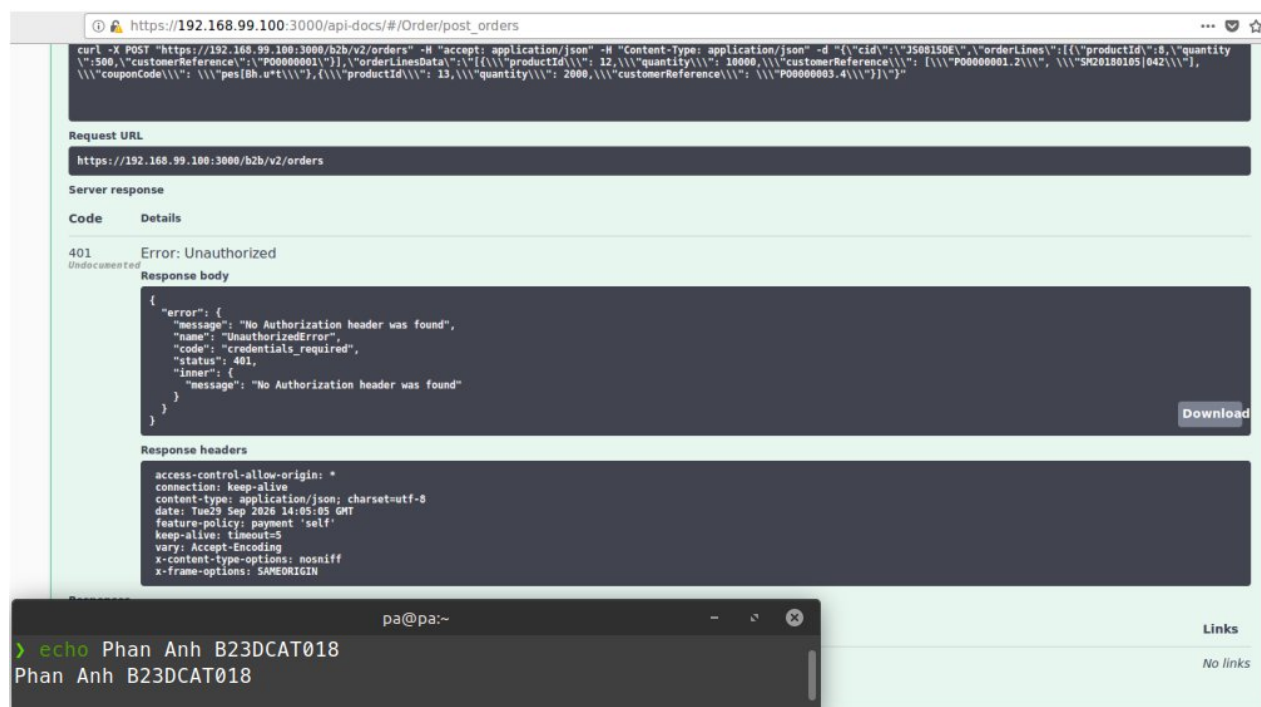
Mục tiêu: Sinh viên khai thác lỗ hổng Insecure Deserialization bằng cách gửi dữ liệu JSON độc hại qua yêu cầu POST, từ đó kiểm tra xem máy chủ có thực thi mã ngoài ý muốn hay không. Lỗ hổng xuất hiện do ứng dụng không kiểm tra và xử lý an toàn dữ liệu đầu vào trước khi thực thi, tạo điều kiện cho kẻ tấn công chèn mã độc, gây ảnh hưởng đến hoạt động của hệ thống hoặc chiếm quyền điều khiển máy chủ.

Bước 1: Mở trình duyệt và truy cập: <https://192.168.99.100:3000/api-docs>. Đây là tài liệu mô tả các API của ứng dụng. Trong danh sách API, sinh viên tìm đến: Order → POST /orders. Nhấp vào API này để xem chi tiết.



Hình 14 Xem chi tiết POST

Bước 2: Sinh viên nhấn Try it out, không thay đổi nội dung mặc định và thực hiện nhấn Execute. Khi ấy máy chủ trả về thông tin: 401 Unauthorized. Điều này cho thấy API yêu cầu người dùng phải đăng nhập và gửi kèm Bearer Token.

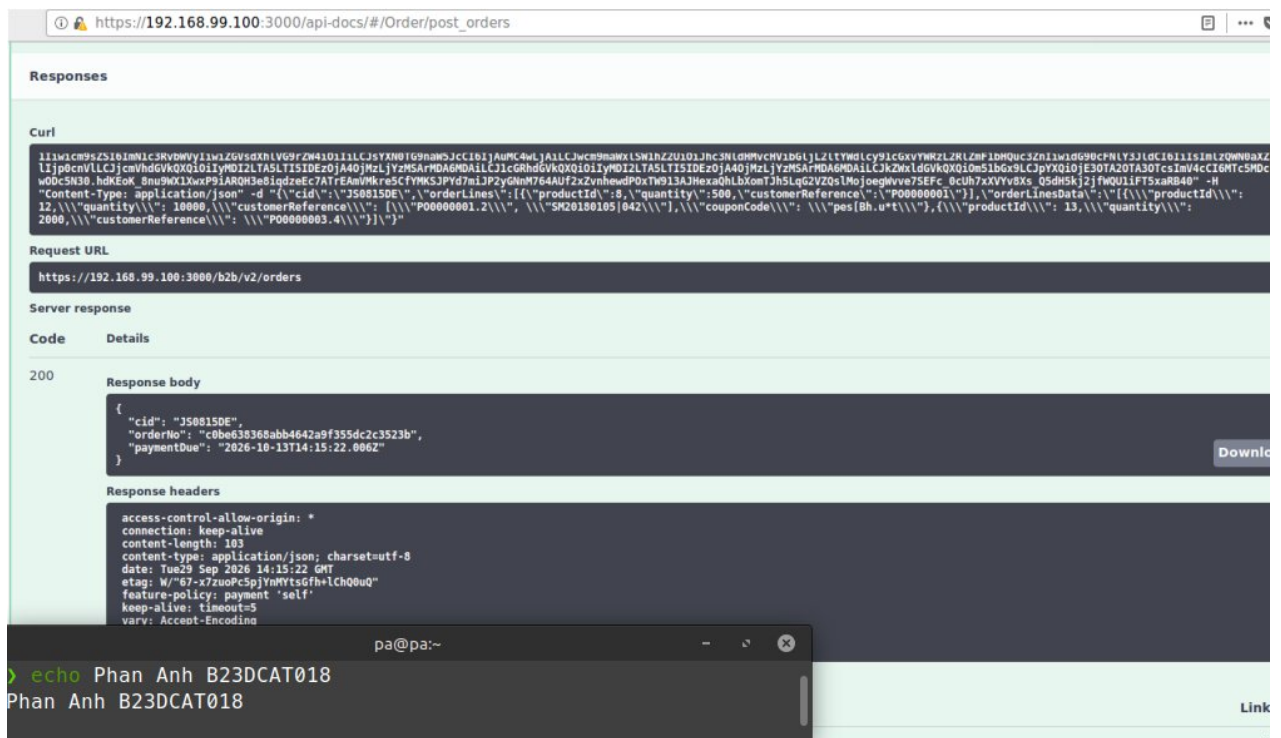


Hình 15 Bấm thử Try it out

[illegible]

Bước 4: Sinh viên quay lại trang web: <https://192.168.99.100:3000/api-docs>. Sau đó thực hiện nhấn vào Authorize và dán Bearer Token vừa sao chép. Tiếp theo chọn Post /orders → Try it out để kiểm tra lại xem API sau xác thực đã thành công chưa.



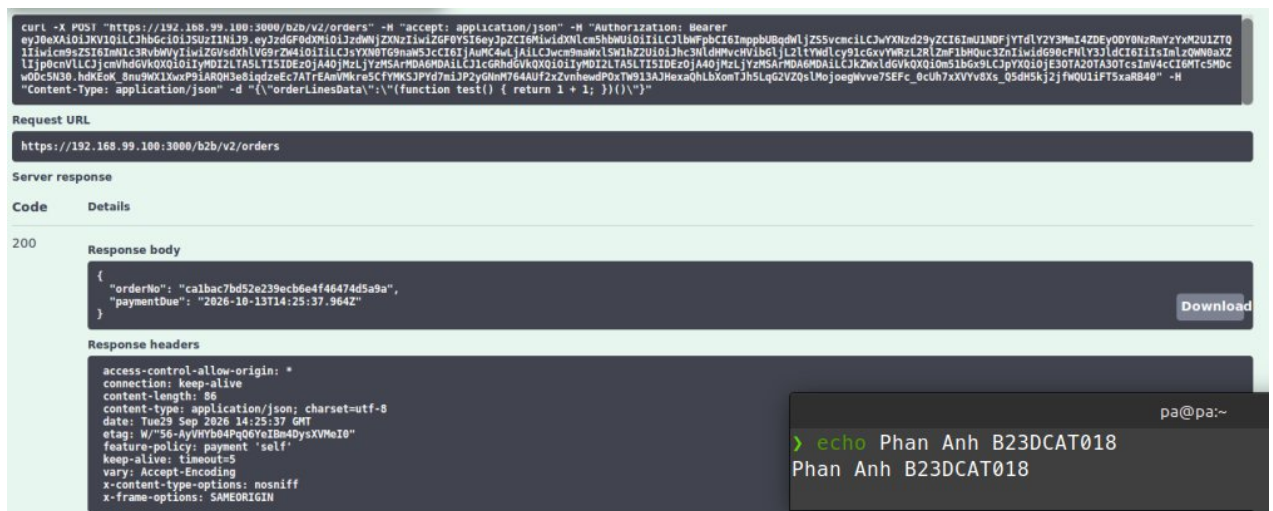


Hình 18 API được xác thực thành công

Bước 5: Ứng dụng xử lý dữ liệu trong trường orderLinesData không an toàn. Dẫn đến, nếu chuỗi JSON chứa một đoạn mã JavaScript, máy chủ sẽ thực thi đoạn mã này trong quá trình xử lý. Trong phần Request Body, thay toàn bộ nội dung bằng:

```
{ "orderLinesData": "(function dos() { while(true); })()" }
```

Sau đó nhấn Execute.



Hình 19 Execute thành công

Máy chủ phản hồi 200 OK sau khoảng thời gian trễ, xác nhận payload gây vòng lặp vô hạn đã được thực thi thành công.

Câu hỏi 2:

Vì sao việc deserialize trực tiếp một chuỗi JSON thành mã JavaScript thực thi được lại là

một lỗ hổng nghiêm trọng? Đề xuất biện pháp phòng chống tương ứng.

Trả lời: Việc deserialize trực tiếp một chuỗi JSON thành mã JavaScript thực thi được là lỗ hổng nghiêm trọng vì dữ liệu người dùng gửi lên có thể bị biến thành mã lệnh chạy trên máy chủ. Kẻ tấn công có thể lợi dụng để gây treo dịch vụ, tiêu tốn tài nguyên hệ thống, đọc dữ liệu nhạy cảm hoặc thực hiện hành vi ngoài ý muốn. Trong bài thực hành, payload vòng lặp vô hạn khiến server phản hồi lỗi Infinite loop detected, cho thấy dữ liệu trong orderLinesData đã được server xử lý như mã lệnh.

Biện pháp phòng chống là không sử dụng eval, Function hoặc cơ chế thực thi mã động với dữ liệu đầu vào. Server cần parse JSON bằng phương thức an toàn, kiểm tra schema dữ liệu, giới hạn kiểu dữ liệu và độ dài đầu vào, chỉ cho phép các trường hợp hợp lệ theo whitelist, đồng thời áp dụng timeout và cơ chế kiểm soát tài nguyên để tránh bị tấn công từ chối dịch vụ.

6.2 - Gây từ chối dịch vụ bằng ReDoS

Trong nhiệm vụ này, sinh viên sẽ tìm hiểu cách dữ liệu đầu vào không được kiểm tra an toàn có thể khiến máy chủ thực thi một biểu thức chính quy (Regular Expression) nguy hiểm, đây là một dạng tấn công có thể làm máy chủ tiêu tốn rất nhiều tài nguyên xử lý. Sinh viên tận dụng quá trình ở nhiệm vụ trước và thực hiện thay Request Body thành:

```
{ "orderLinesData": "/((a+)+)b/.test('aaaaaaaaaaaaaaaaaaaaaaaaaaaaa')"
```

Sau đó nhấn Execute.

The screenshot displays a web application security tool interface. The 'Curl' tab shows a POST request to `https://192.168.99.100:3000/b2b/v2/orders` with a payload designed to trigger a ReDoS attack. The 'Server response' tab shows a 503 Service Unavailable error. A terminal window on the right shows the command `echo Phan Anh B23DCAT018` being executed.

Hình 20 Thử redos

Câu hỏi 3:

Sau một khoảng thời gian đợi máy chủ xử lý, máy chủ sẽ phản hồi ra thông tin gì? Giải thích lý do?

Trả lời:

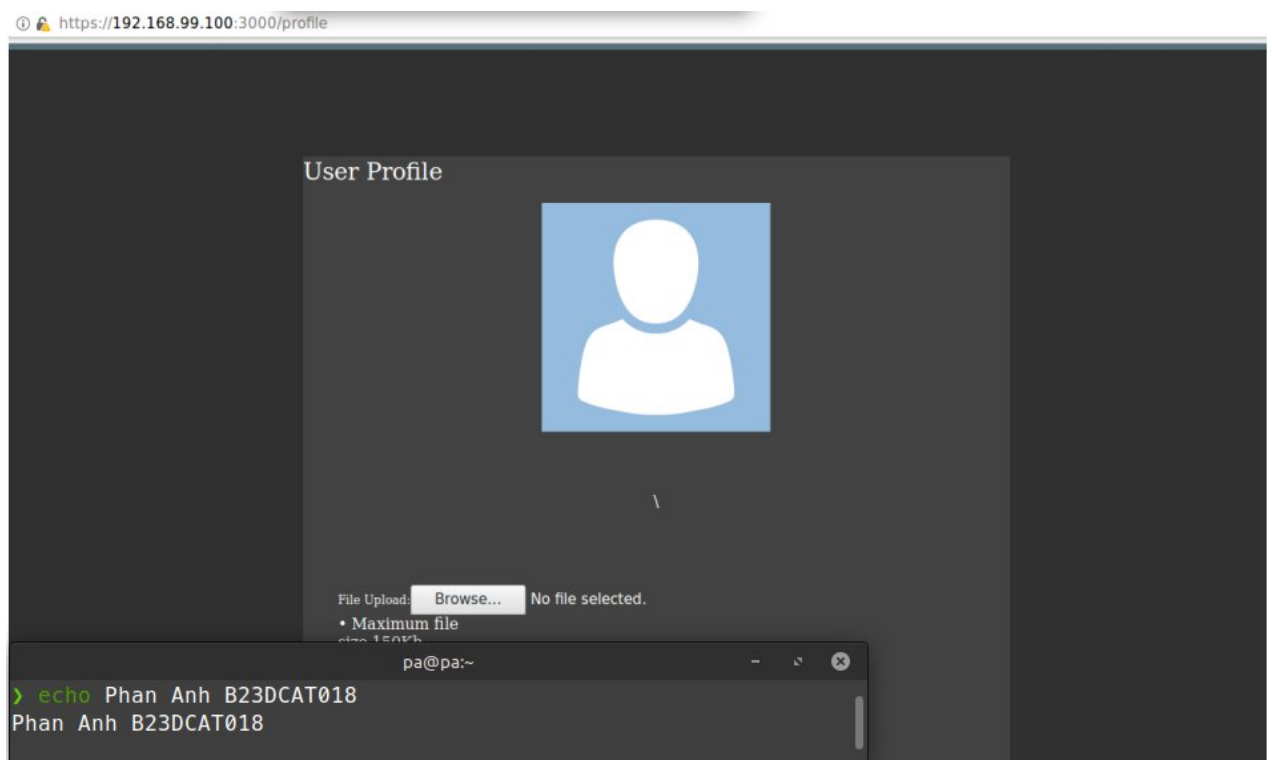
Máy chủ phản hồi lỗi 503 Service Unavailable với thông báo tạm thời không khả dụng. Nguyên nhân là biểu thức chính quy `/((a+)+)b/` xử lý chuỗi nhiều ký tự a nhưng không có

b, gây hiện tượng catastrophic backtracking, làm server tiêu tốn nhiều tài nguyên và phản hồi chậm hoặc lỗi.

Nhiệm vụ 7 - Khai thác lỗ hổng SSTI để đọc dữ liệu nhạy cảm trên máy chủ

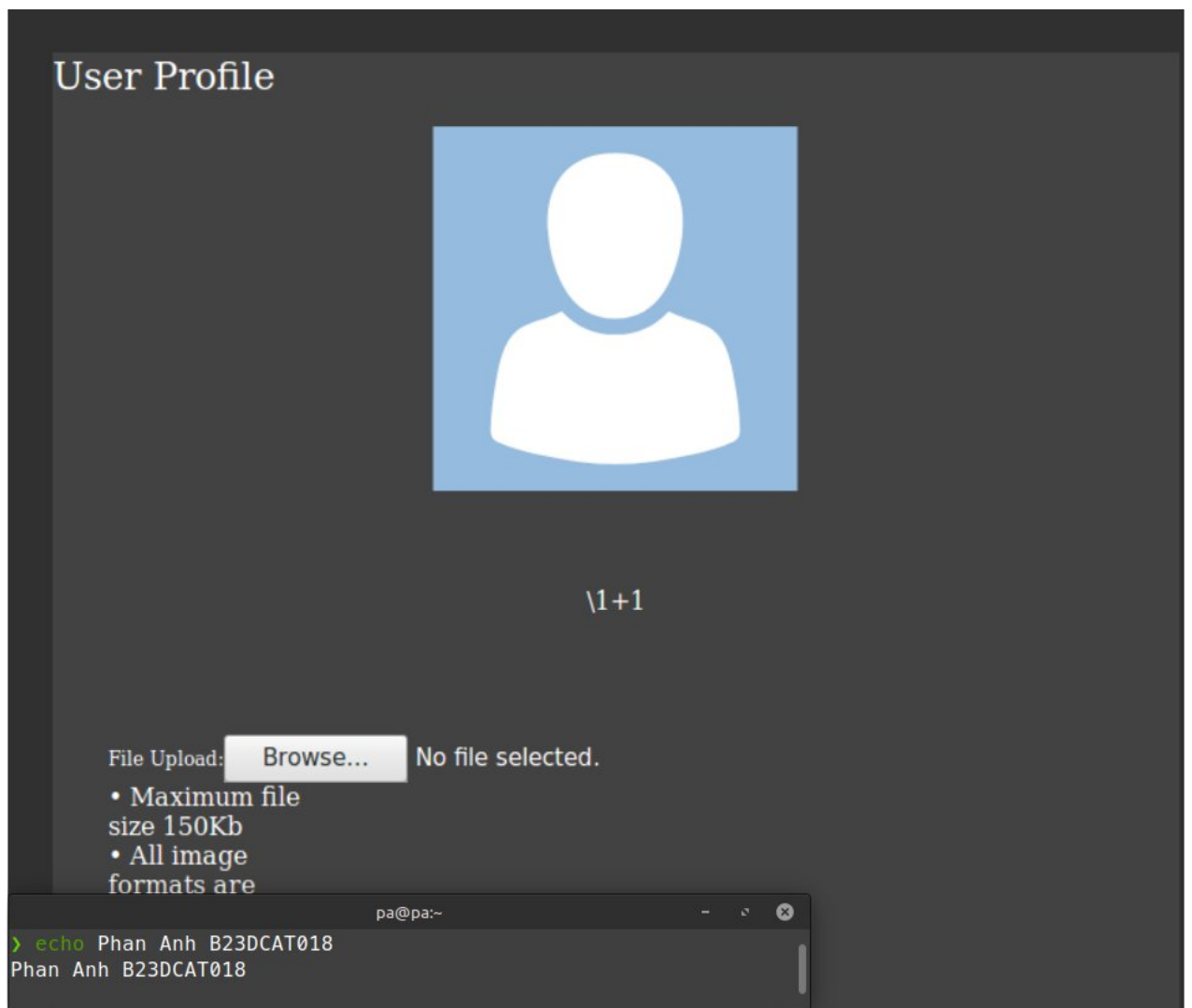
Mục tiêu: Mục tiêu của nhiệm vụ này là tìm hiểu cách lỗ hổng Server-Side Template Injection (SSTI) có thể bị khai thác để buộc máy chủ thực thi mã tùy ý. Trong bài thực hành này, sinh viên sẽ xác minh sự tồn tại của lỗ hổng SSTI trên trang hồ sơ người dùng, sau đó sử dụng payload phù hợp để hoàn thành thử thách.

Bước 1: Sinh viên đăng nhập vào ứng dụng bằng một tài khoản bất kỳ. Sau đó mở trang: <https://192.168.99.100:3000/profile>. Khi quan sát địa chỉ này, có thể nhận thấy trang hồ sơ được xây dựng bằng Pug Template Engine, vì vậy đây là vị trí có khả năng tồn tại lỗ hổng Server-Side Template Injection (SSTI).



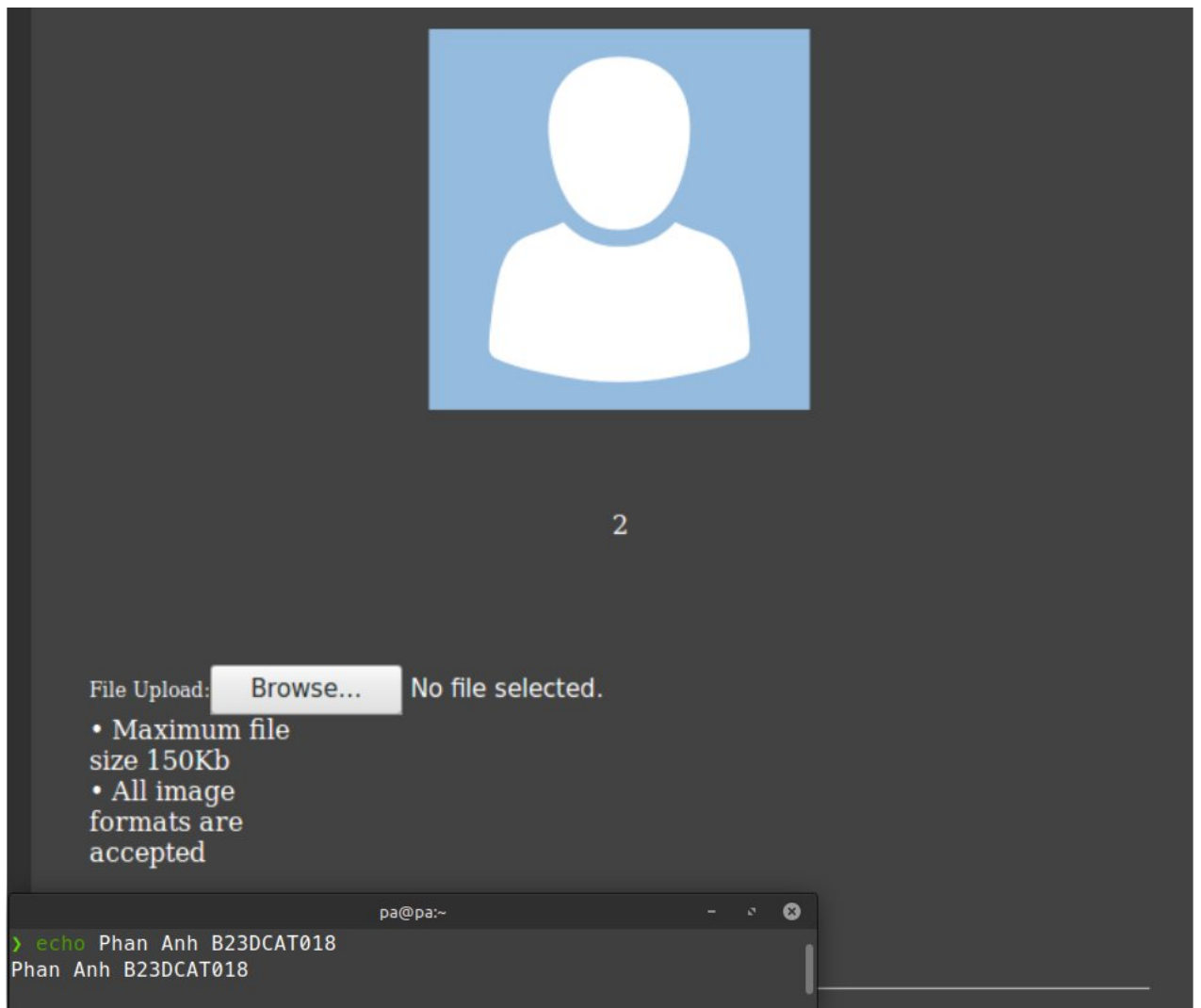
Hình 21 Trang profile

Bước 2: Sinh viên nhập tên người dùng: 1+1. Sau đó nhấn Set Username. Khi ấy, tên người dùng sẽ được hiển thị đúng nguyên văn là: 1+1. Điều này cho thấy dữ liệu đầu vào chưa được xử lý như một biểu thức của Template Engine.



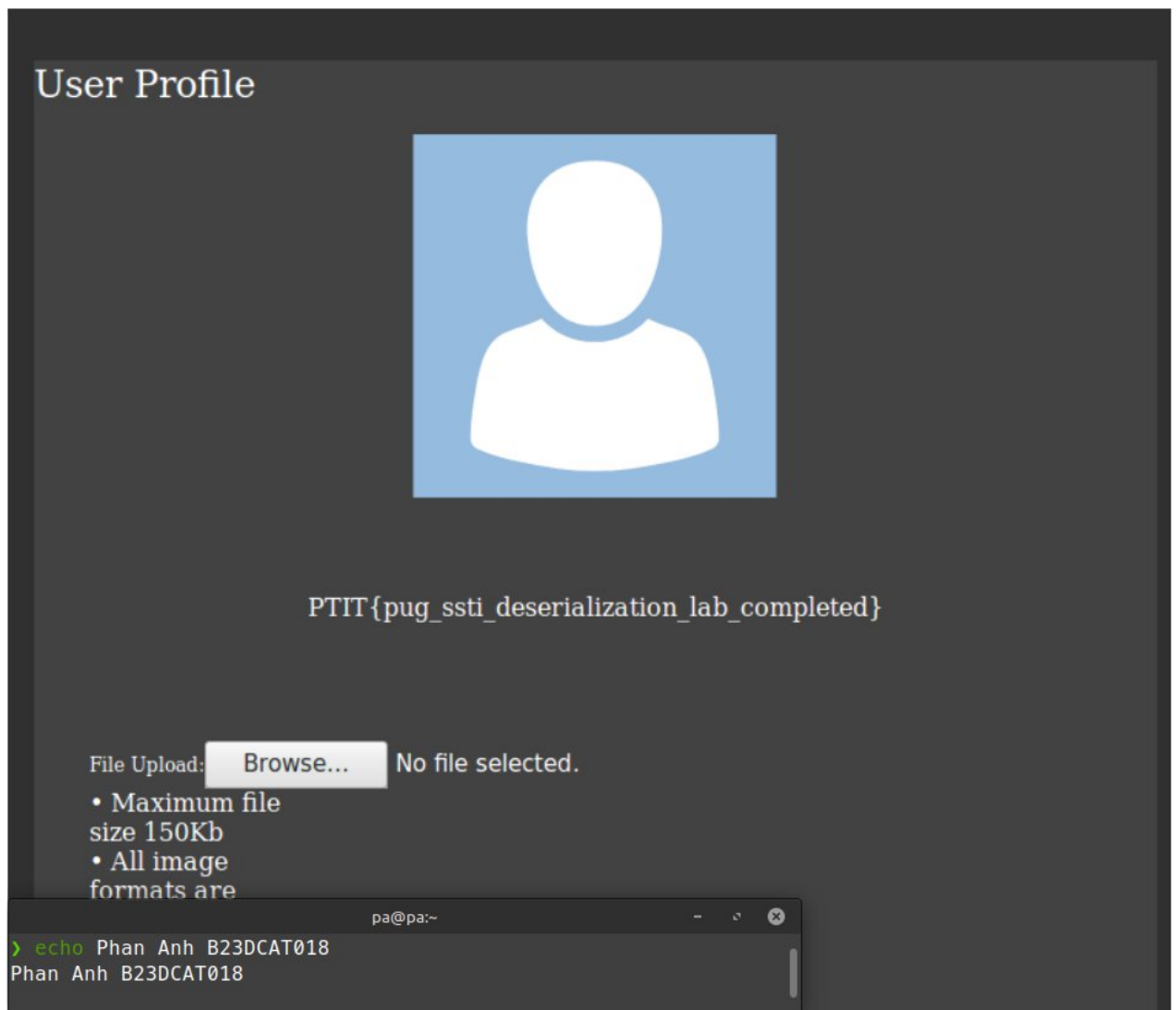
Hình 22 Set thử Username

Bước 3: Sinh viên thay tên người dùng bằng: `#\1+1`. Sau đó nhấn Set Username. Khi ấy, tên người dùng sẽ hiển thị là: 2. Điều này chứng tỏ biểu thức trong Template đã được máy chủ thực thi và ứng dụng tồn tại lỗ hổng Server-Side Template Injection (SSTI).



Hình 23 Set Username thành #{1+1}

Bước 4: Trên máy chủ có giấu một tệp tin bí mật (flag.txt) mà sinh viên cần tìm cách đọc được nội dung. Dựa trên nguyên lý tương tự Bước 3, sinh viên xây dựng một biểu thức Pug lợi dụng đối tượng global.process của Node.js để gọi các hàm đọc file của hệ thống, sau đó nhập biểu thức đó vào ô tên người dùng và nhấn Set Username để quan sát nội dung tệp flag được trả về.



Hình 24 Xác định được flag

Xác minh thành công lỗ hổng SSTI trên trang hồ sơ và đọc được nội dung tệp nhạy cảm trên máy chủ thông qua payload SSTI.

Câu hỏi 4:

Vì sao trang hồ sơ sử dụng Pug Template Engine lại tồn tại lỗ hổng SSTI, và biện pháp nào giúp ngăn chặn việc dữ liệu người dùng bị Template Engine thực thi như một biểu thức?

Trả lời: Trang hồ sơ tồn tại lỗ hổng SSTI vì dữ liệu người dùng nhập vào được đưa vào Pug Template Engine theo cách không an toàn. Khi nhập `#{1+1}`, server không hiển thị nguyên văn mà xử lý nội dung này như một biểu thức template, dẫn đến kết quả trả về là 2. Điều này cho thấy attacker có thể chèn biểu thức nguy hiểm để truy cập đối tượng hệ thống hoặc đọc file nhạy cảm trên máy chủ.

Biện pháp phòng chống là không đưa trực tiếp dữ liệu người dùng vào template dưới dạng mã thực thi, cần escape output trước khi hiển thị, validate dữ liệu đầu vào theo whitelist, chặn các cú pháp template nguy hiểm như `#{...}` và không cho phép template truy cập các đối tượng nguy hiểm như `global.process`.

Kết thúc bài lab và checkwrok

```
student@LabtainerVMware:~/labtainer/labtainer-student$ checkwork
Results stored in directory: /home/student/labtainer_xfer/web-insdes
Successfully copied 26MB to web-insdes-igrader:/home/instructor/B23DCAT018.web-insdes.lab
Successfully copied 2.05kB to /home/student/labtainer_xfer/web-insdes
Labname web-insdes

Student: B23DCAT018
Y - zap_report
Y - deserial_dos
Y - ssti_flag
What is automatically assessed for this lab:

_zap_report: Check if student generated the ZAP traversal report
_deserial_dos: Check if student successfully executed the Insecure Deserialization DoS exploit
_ssti_flag: Check if student successfully executed the Pug SSTI exploit to read the flag
```

Hình 25 Checkwork hoàn thành bài lab